

NYCU Deep Learning Spring 2025

Lab6 Report

Generative Models – conditional DDPM

By 313554044 黃梓誠

授課教師：彭文孝、陳永昇、謝秉均

開課單位：資科工碩

永久課號：CSIC30153

Date Submission: May5, 2025

目錄

1. Introduction (5%).....	2
2. Implementation Details (25%).....	3
2.1 Dataset & Preprocessing.....	3
2.2 Define your multi-label condition embedding method.....	3
2.3 Model Architecture - UNet2DConditionModel.....	3
2.4 Diffusion Scheduler	4
2.5 Training Strategy	4
2.6 Evaluation Function.....	4
3. Results & Discussion (30%)	4
3.1 Show your synthetic image grids (total 16%: 8% * 2 testing data).....	4
3.2 denoising process image(4%)	5
3.3 Discussion of your extra implementations or experiments (10%).....	5
4. Experimental results (40%).....	7
4.1 Classification accuracy on test.json and new test.json (40%)	7
4.2Show your accuracy screenshots (20% * 2 testing data)	7
5. References.....	7

1. Introduction (5%)

本實驗旨在實作一個 conditional Denoising Diffusion Probabilistic Model (Conditional DDPM)，能根據多標籤條件 (Multi-label conditions) 生成對應的合成影像。模型輸入為一組物件條件 (例如："red sphere", "cyan cube")，根據該條件能產生包含這些物件的圖像，並透過預訓練的 ResNet18-based Evaluator 分類器進行準確率評估。

2. Implementation Details (25%)

2.1 Dataset & Preprocessing

本次實驗採用 iCLEVR dataset，影像資料為 RGB 格式，資料包含 train.json, test.json, new_test.json，以及定義 24 類物件的 objects.json，配合 Multi-label conditions 的生成需求。

因此我實作 iCLEVRDataset Class，以便自動辨識資料格式建立影像對應標籤。再依據 object_map 將多標籤轉換為 one-hot 向量(將對應物件位置設為 1.0)

```
1.     label_vector[self.object_map[obj_name]] = 1.0
```

最後利用 PIL.Image 讀取轉換為 Tensor，使用 transform 進行 Resize 與 Normalize([0.5, 0.5, 0.5], [0.5, 0.5, 0.5]) 處理以符合預訓練分類器需求。

2.2 Define your multi-label condition embedding method

我設計了一個 ConditionEmbedder，將多標籤 one-hot 向量投影至固定維度 latent space，維度設為 128，後續會送入 UNet 中作為 Cross-Attention 的輸入。結構如下：

```
1. self.mlp = nn.Sequential(  
2.     nn.Linear(num_classes, hidden_dim),  
3.     nn.SiLU(),  
4.     nn.Linear(hidden_dim, embed_dim)  
5. )
```

2.3 Model Architecture - UNet2DConditionModel

使用 HuggingFace diffusers 提供的 UNet2DConditionModel 作為 backbone。模型設定如下：

輸入與輸出為 3-channel 圖像 (RGB)，影像大小為 64x64

模型各 Layer 使用：

Down Block: ["DownBlock2D", "CrossAttnDownBlock2D",
"CrossAttnDownBlock2D"]

Up Block: ["CrossAttnUpBlock2D", "CrossAttnUpBlock2D", "UpBlock2D"]

Mid Block: "UNetMidBlock2DCrossAttn"

各層 Block Output Channels 設為 (64, 128, 256) Attention 維度設為 128，與條件嵌入一致

2.4 Diffusion Scheduler

採用 DDPM Scheduler 設定 noise schedule，選用 linear β schedule，diffusion 步數設為 1500 比原始論文多出 500，目的是希望他可以有更多 denoise 時間，每次訓練將會隨機取一個 timestep t 加入高斯雜訊後，學習預測 noise。

```
noise_scheduler = DDPMScheduler(  
    num_train_timesteps=args.num_train_timesteps,  
    beta_schedule=args.beta_schedule,  
)
```

2.5 Training Strategy

Optimizer: AdamW (設定 $\beta_1=0.9$, $\beta_2=0.999$, weight decay= $1e-2$)

Learning Rate: $3e-5$ ，並使用 Linear Scheduler 搭配 Warm-up 步驟 (500 步)

使用 Mixed Precision (torch.amp.autocast) 與 Gradient Scaler 降低 memory 使用
梯度累積每 2 step 執行一次 optimizer 更新

2.6 Evaluation Function

評估資料來自 test.json 與 new_test.json，輸出圖片經過 Denormalize 後，用 Evaluator 進行評分設計若準確率高於 0.8，將儲存 Image Grid 與 Denoising 過程並記錄樣本準確率後回傳 Mean Accuracy

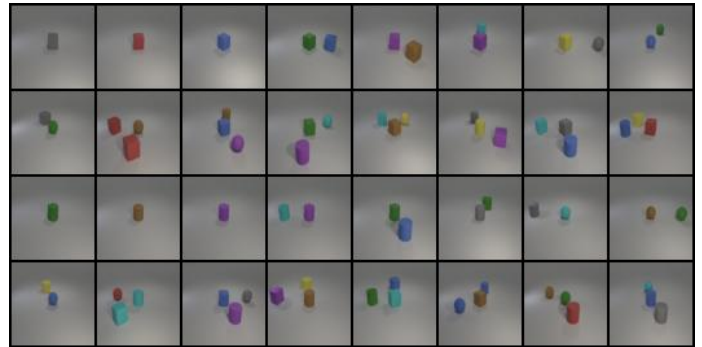
3. Results & Discussion (30%)

3.1 Show your synthetic image grids (total 16%: 8% * 2 testing data)

test.json 與 new_test.json 皆產出 32 張圖像，使用 make_grid() from torchvision.utils 建構 8x4 的 grids



The synthetic image grid on new_test.json



The synthetic image grid on test.json

3.2 denoising process image with the label set ["red sphere", "cyan cylinder", "cyan cube"] (4%)

使用取樣過程中的 10 步顯示從完全噪音到清晰圖像的變化，並使用 `make_grid()` 將過程視覺化成 1x10 的圖像序列

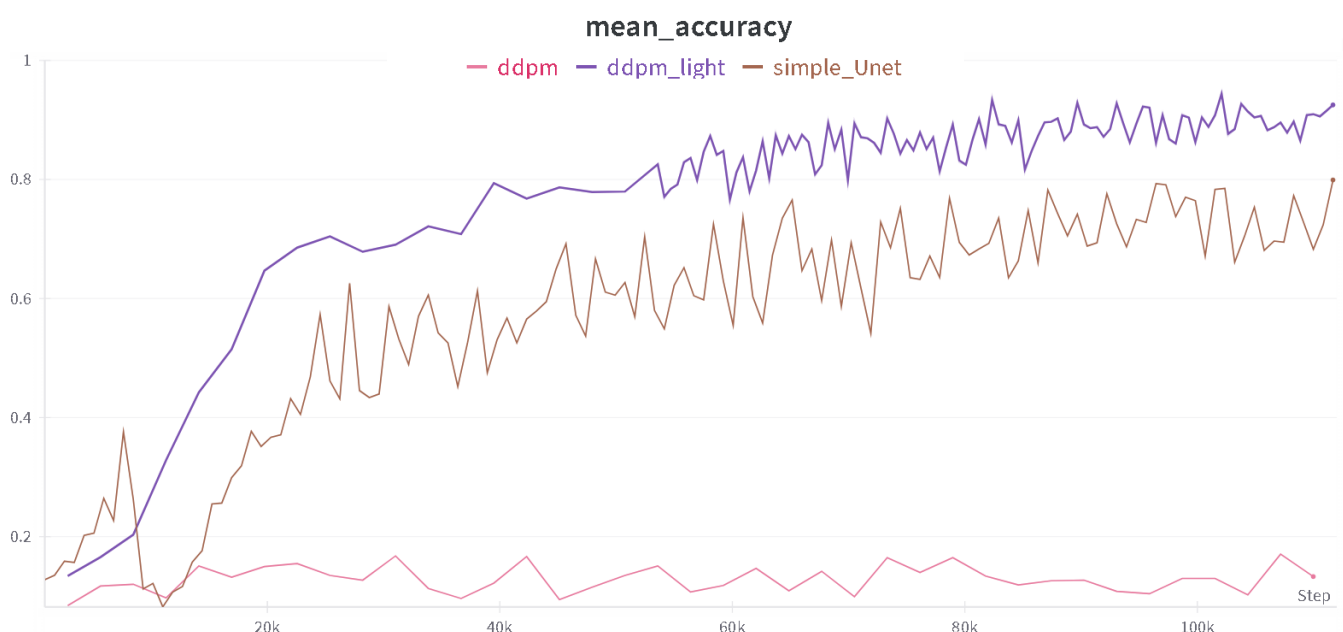


Denoising process image

3.3 Discussion of your extra implementations or experiments (10%)

本次實驗除主線任務外，共設計並對比三種架構的實作方式對於最終效果有何影響，分別為

1. `ddpm_light.py` (`ddpm.py` 之輕量版 為 report 使用之版本)
2. `ddpm.py` (使用 HuggingFace `UNet2DConditionModel`)
3. `simple_Unet.py` (手刻 UNet + attntion)。



Appendix : extra implementations.png

針對三者訓練穩定性與準確率表現進行分析如下：

首先，`ddpm_light.py` 成功達成最高測試準確率 (test accuracy 0.96+)，訓練過程極為穩定，幾無震盪，顯示該模型之結構與訓練策略與 iCLEVR 資料集相容性良好。推測主因在於模型架構較淺，參數量小(相比於 `ddpm.py`)，能在有限數據下有效收斂而不過擬合，適合本任務之 scale。

相對地，ddpm.py 雖同樣使用 HuggingFace 官方的 UNet2DConditionModel，具備完整的 Cross-Attention 機制與深層卷積架構，理論上應能強化條件建模能力。然而實驗中訓練效果不佳，acc 長期震盪於 0.1~0.2 間，無明顯提升，推測原因可能為：

1. 模型過於複雜導致梯度不穩定；
2. Condition Embedding 與 UNet 模型設計對接上有錯誤不相容；

此外，simple_Unet.py 版本雖無複雜注意力結構，僅為標準 encoder-decoder 架構，但經實測仍可達到 0.88 左右準確率，表現穩定性中等但具學習能力，驗證即便無 sophisticated block，仍能捕捉資料分布特徵。

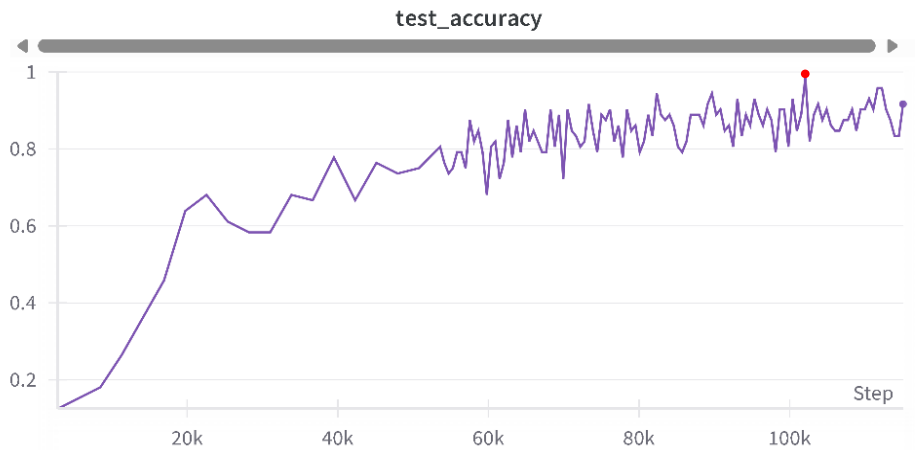
按照上述分析，本次實驗成功驗證了 Conditional DDPM 能透過 attention 機制達成穩定的多標籤圖像生成。同時不同架構的分析讓我了解到：對於小型條件生成資料集，**模型的設計應適度簡化以防止學習不穩，若未來擴展至更複雜資料或更強生成需求，可以再適度調整層數**，或是實作 Classifier-Free Guidance 或 fine-tune 專屬 condition encoder，以達成穩定與泛化之平衡。

比較項目	ddpm	ddpm_light	simple_Unet
Model 基底	HuggingFace UNet2DConditionModel	HuggingFace UNet2DConditionModel	手刻 SimpleConditionalUNet
Block output channels	(128, 256, 512)	(64, 128, 256)	Encoder: 64 → 128 → 256 Decoder 對稱
每層 ResNet layers	layers_per_block = 2	layers_per_block = 1	每個 Residual Block 含 2 Conv 共 7 個 ResBlock
Down Block 組合	DownBlock2D → CrossAttnDownBlock2D × 2	同左	3ResBlock + AvgPool (× 2)
Up Block 組合	CrossAttnUpBlock2D × 2 → UpBlock2D	同左	3 ResBlock + Bilinear Upsample (× 2)
Mid Block	UNetMidBlock2DCrossAttn	UNetMidBlock2DCrossAttn (較小)	自訂 Bottleneck ResBlock
Attention	Cross-attention (中段+上下採樣層皆有)	同上但通道較少	2self-attention blocks (256ch & 128ch)
Cond Embedding	MLP → 128 dim，經 Cross-Attention 注入	同左	MLP → 256 dim
Time Embedding	Diffusers 內建 sinusoidal + MLP	同左	Sinusoidal(256) + MLP

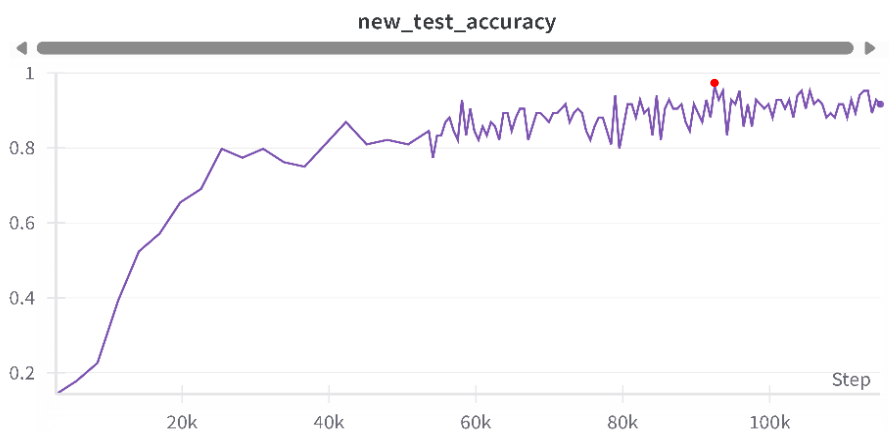
4. Experimental results (40%)

4.1 Classification accuracy on test.json and new test.json (40%)

Best test.json accuracy: **0.98611**(Training epoch 362)



Best new_test.json accuracy: **0.9762**(Training epoch 460)



4.2 Show your accuracy screenshots (20% * 2 testing data)

```
file/test.json accuracy = 0.9861
```

```
file/new_test.json accuracy = 0.9762
```

5. References

HuggingFace Diffusion Model Class: <https://github.com/huggingface/diffusers>

DDPM 原始論文: [Ho et al., 2020](#)